

Contents

Object Oriented Programming Languages	1
Complete Topic List	1
Chapter Files	1
1.1 Classes and Objects, Instantiation	2
1.2 Comparing and Copying Objects	9
1.3 Testing	15
Cheat-Sheet	17
Review Checklist	18

Object Oriented Programming Languages

Complete Topic List

1. Classes and Objects

- Classes and objects, instantiation
- Comparing and copying of objects
- Testing

2. Encapsulation and Members

- Encapsulation, members, and constructors
- Instance and static members
- Information hiding and visibility modifiers

3. Polymorphism and Type Systems

- Overloading
- Inheritance and multiple inheritance
- Subtyping, static and dynamic types, and type checking
- Overriding and dynamic binding
- Interfaces
- Generics
- Subtype and parametric polymorphism

4. Memory Management

- Memory management and garbage collection

5. Program Structure

- Program structure, packages, and compilation units

Chapter Files

- `chapters/chap1_classes_objects.md`
- `chapters/chap2_encapsulation_members.md`

- chapters/chap3_polymorphism_types.md
- chapters/chap4_memory_management.md
- chapters/chap5_program_structure.md # Chapter 1: Classes and Objects

1.1 Classes and Objects, Instantiation

Definition / Theoretical Background

Class: A blueprint or template that defines the structure and behavior of objects. It specifies: -

Fields (attributes/properties): Data that objects store - **Methods** (behaviors): Operations that objects can perform

Object: An instance of a class - a concrete entity created from the blueprint.

Concept	What It Is	Example
Class	Blueprint	<code>class Dog { ... }</code>
Object	Instance of class	<code>Dog buddy = new Dog();</code>
Instantiation	Creating object from class	<code>new Dog()</code>
State	Values in object's fields	<code>buddy.name = "Buddy"</code>
Identity	Unique identifier	Each object has unique address
Behavior	Methods object can perform	<code>buddy.bark()</code>

Class vs Object Visualization

CLASS (Blueprint)

```
+-----+
| class Dog {
|   String name;
|   int age;
|   void bark() { ... }
| }
|
| Doesn't exist in memory
+-----+
```

OBJECT (Instance)

```
+-----+
| Dog buddy = new Dog()
|
| name = "Buddy"
| age = 3
| bark()
|
| Lives in HEAP memory
+-----+
```

You can create MANY objects from ONE class!

```
Dog buddy = new Dog();
Dog max = new Dog();
Dog charlie = new Dog();

+-----+
| name = "Buddy"
| age = 3
+-----+

Dog max = new Dog();
+-----+
| name = "Max"
| age = 5
+-----+

Dog charlie = new Dog();
+-----+
```

```
| name = "Charlie"|
| age = 2          |
+-----+
```

Declaration and Instantiation

```
// Class definition
class Dog {
    // Instance variables (fields)
    String name;
    int age;

    // Constructor
    Dog(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Methods
    void bark() {
        System.out.println(name + " says Woof!");
    }
}

// Instantiation and use
public class Main {
    public static void main(String[] args) {
        Dog buddy = new Dog("Buddy", 3); // Create object
        Dog max = new Dog("Max", 5);

        buddy.bark(); // "Buddy says Woof!"
        max.bark(); // "Max says Woof!"

        // Objects have independent state
        buddy.age = 4; // Only buddy's age changes
        System.out.println(max.age); // Still 5!
    }
}
```

Java

```
# Class definition (Python)
class Dog:
    # Constructor (special method)
    def __init__(self, name, age):
        self.name = name # self refers to the object
```

```

        self.age = age

    def bark(self):
        print(f"{self.name} says Woof!")

# Instantiation
buddy = Dog("Buddy", 3)
max = Dog("Max", 5)

buddy.bark()    # "Buddy says Woof!"
max.bark()      # "Max says Woof!"

```

Python

```

class Dog {
public:
    string name;    // Instance variables
    int age;

    // Constructor
    Dog(string name, int age) : name(name), age(age) {}

    void bark() {
        cout << name << " says Woof!" << endl;
    }
};

int main() {
    Dog* buddy = new Dog("Buddy", 3); // Heap allocation
    Dog max("Max", 5);                // Stack allocation

    buddy->bark(); // "Buddy says Woof!"
    max.bark();   // "Max says Woof!"

    delete buddy; // Don't forget to free heap memory!
}

```

C++

The new Keyword (Instantiation)

```
Dog buddy = new Dog(...);
```

What new does: 1. Allocates memory for the object (usually on the heap) 2. Initializes fields to default values (0, null, false) 3. Calls the constructor method 4. Returns a reference to the object

```
new Dog("Buddy", 3)
```

```

      |
      v
+-----+
| 1. Allocate memory (heap) |
| 2. Initialize fields:    |
|   name = null           |
|   age = 0               |
| 3. Call constructor:    |
|   name = "Buddy"        |
|   age = 3               |
| 4. Return reference (address) |
+-----+

```

Default Values

```

class Example {
    int num;           // Default: 0
    double decimal;    // Default: 0.0
    boolean flag;      // Default: false
    String text;       // Default: null
    int[] arr;         // Default: null
    Object obj;        // Default: null

    // Constructor to initialize
    Example() {
        num = 10; // Now it's 10
        // text still null if not initialized
    }
}

```

Default values table:

Type	Default Value
byte, short, int, long	0
float, double	0.0
char	'�0000' (null character)
boolean	false
Object reference	null

Memory Where Objects Live

```

STACK (local variables)      HEAP (objects)
+-----+                   +-----+
| Dog buddy ----->| Dog object |
| (reference)       | | name: "Buddy" |
+-----+           | | age: 3       |
                   +-----+

```

In Java, Python, C#: - **Objects live on the Heap** (managed memory) - **References live on the Stack** (variables point to heap)

In C++: - Objects can live on **Stack** (if declared locally) or **Heap** (if **new**)

Edge Cases & Traps

Trap 1: Reference vs Object Confusion

```
Dog d1 = new Dog("Buddy", 3);
Dog d2 = d1; // Copies REFERENCE, not object!

// Now d1 and d2 point to SAME object!
d2.name = "Max";
System.out.println(d1.name); // "Max"! Changed because same object!

// For independent copies, need to copy explicitly:
Dog d3 = new Dog(d1.name, d1.age); // Creates NEW object
```

Trap 2: Null Reference

```
Dog d; // d is null (no object!)
d.bark(); // NullPointerException! Object doesn't exist!

// Must initialize first:
d = new Dog("Buddy", 3);
d.bark(); // Now works
```

Trap 3: Uninitialized Fields

```
class Example {
    int x; // Default 0, not uninitialized garbage!
}
// x is 0, not random garbage
// Unlike C/C++ local variables!
```

Trap 4: Class is a Type, Object is an Instance

```
Dog d = new Dog(); // Dog is type/class, d is instance
// You cannot do: new Dog.bark() (wrong!)
// Must: d.bark() (using object instance)
```

Examples

```
class BankAccount {
    // Fields
    String accountNumber;
    String ownerName;
    double balance;

    // Constructor
```

```

BankAccount(String accNum, String name, double initialBalance) {
    accountNumber = accNum;
    ownerName = name;
    balance = initialBalance;
}

// Methods
void deposit(double amount) {
    if (amount > 0) {
        balance += amount;
    }
}

void withdraw(double amount) {
    if (amount <= balance) {
        balance -= amount;
    } else {
        System.out.println("Insufficient funds!");
    }
}

void display() {
    System.out.println(ownerName + ": $" + balance);
}
}

public class Main {
    public static void main(String[] args) {
        BankAccount acc1 = new BankAccount("12345", "Alice", 1000);
        BankAccount acc2 = new BankAccount("67890", "Bob", 500);

        acc1.deposit(500);    // Alice: $1500
        acc2.withdraw(100);   // Bob: $400
        acc1.display();       // Alice: $1500.0
        acc2.display();       // Bob: $400.0

        // Each object has its OWN state
        System.out.println(acc1.balance); // 1500.0
        System.out.println(acc2.balance); // 400.0
    }
}

```

Example 1: Bank Account Class

```

class Rectangle:
    def __init__(self, width, height):

```

```

        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

    def scale(self, factor):
        self.width *= factor
        self.height *= factor

# Create two rectangle objects
r1 = Rectangle(5, 3)
r2 = Rectangle(4, 2)

print(f"r1 area: {r1.area()}") # 15
print(f"r2 area: {r2.area()}") # 8

r1.scale(2) # Scale r1 by 2
print(f"r1 area after scale: {r1.area()}") # 60
print(f"r2 unchanged: {r2.area()}") # Still 8!

```

Example 2: Rectangle Class

```

class Student {
    String name;
    int age;
    String major;

    // Constructor 1: Full
    Student(String name, int age, String major) {
        this.name = name;
        this.age = age;
        this.major = major;
    }

    // Constructor 2: Default major
    Student(String name, int age) {
        this.name = name;
        this.age = age;
        this.major = "Undeclared";
    }

    // Constructor 3: Default values

```

```

Student() {
    this.name = "Unknown";
    this.age = 0;
    this.major = "Undeclared";
}
}

// Usage
Student s1 = new Student("Alice", 20, "CS");           // Full
Student s2 = new Student("Bob", 19);                  // Default major
Student s3 = new Student();                           // All defaults

```

Example 3: Constructor Overloading (Multiple Constructors)

1.2 Comparing and Copying Objects

Comparing Objects

Reference Equality vs Value Equality

Type	What It Checks	Syntax
Reference equality	Same memory location?	<code>==</code> in Java for references
Value equality	Same content?	<code>.equals()</code> in Java

```

String s1 = new String("hello");
String s2 = new String("hello");

// Reference comparison (are they the same object?)
System.out.println(s1 == s2);           // false (different objects)

// Value comparison (do they have same content?)
System.out.println(s1.equals(s2));       // true (same content!)

// BUT: String literals are pooled!
String s3 = "hello";
String s4 = "hello";

System.out.println(s3 == s4);           // true (same pooled object!)
System.out.println(s3.equals(s4));       // true (same content too)

```

```

class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }
}

```

```

Point p1 = new Point(1, 2);
Point p2 = new Point(1, 2);

System.out.println(p1 == p2);           // false! Different objects!
System.out.println(p1.equals(p2));      // false! equals() inherited from Object
                                         // does reference comparison by default!

// You need to OVERRIDE equals()!
class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true; // Same reference
        if (!(obj instanceof Point)) return false;
        Point other = (Point) obj;
        return this.x == other.x && this.y == other.y;
    }
}

```

Why == Doesn't Work for Value Comparison

```

# Python: == does VALUE comparison by default!
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p1 = Point(1, 2)
p2 = Point(1, 2)
p3 = p1

print(p1 == p2) # True! (Python's == compares values by default)
print(p1 == p3) # True! (Same object, same value)

# is checks identity (like reference equality in Java)
print(p1 is p2) # False! Different objects
print(p1 is p3) # True! Same object

```

Python Comparison

Copying Objects

```

class Address {
    String city;
}

```

```

    Address(String city) { this.city = city; }
}

class Person {
    String name;
    Address address; // Reference type field!

    Person(String name, Address address) {
        this.name = name;
        this.address = address;
    }
}

// Original
Address addr = new Address("NYC");
Person original = new Person("Alice", addr);

// Shallow copy - copies reference, not object!
Person shallowCopy = original;
// shallowCopy.address points to SAME Address object as original!

// Deep copy - copies the referenced objects too!
Person deepCopy = new Person(original.name, new Address(original.address.city));
// deepCopy.address is a NEW object with same values!

```

Shallow Copy vs Deep Copy

Shallow vs Deep Copy Visualization

SHALLOW COPY:

```

original -----+
|               |
| address -----+ |
v               v v
[Address: NYC]    shallowCopy.address -- same object!

```

DEEP COPY:

```

original -----+
|               |
| address -----+ | (original.address)
v               | |
[Address: NYC]   | |
                | |
deepCopy.address -----+ |
|               |
v               v
[Address: NYC]   NEW COPY! Different object

```

```

class Person {
    String name;
    int age;

    // Regular constructor
    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Copy constructor
    Person(Person other) {
        this.name = other.name;
        this.age = other.age;
    }
}

// Using copy constructor
Person original = new Person("Alice", 25);
Person copy = new Person(original); // Creates NEW object with same values!

```

Copy Constructors

```

import copy

class Person:
    def __init__(self, name, address):
        self.name = name
        self.address = address # Could be object reference

# Original
p1 = Person("Alice", "NYC")

# Shallow copy - nested objects shared
p2 = copy.copy(p1)

# Deep copy - nested objects also copied
p3 = copy.deepcopy(p1)

# p1 and p2 are different objects, but their address field might be same object
# p3 is completely independent

```

Python Copy Module

Edge Cases & Traps

Trap 1: == on Objects Compares References, Not Values

```
String a = new String("test");
String b = new String("test");
if (a == b) { // FALSE! Different objects!
    // This block never executes!
}
if (a.equals(b)) { // TRUE! Same content!
    // This block executes!
}
```

[WARNING] **Professor Trap:** Always use `.equals()` for string value comparison!

Trap 2: Mutable Default Arguments (Python)

```
# DANGER! This is a common Python trap!
class BadList:
    def __init__(self, items=[]): # Mutable default!
        self.items = items

lst1 = BadList()
lst1.items.append(1)

lst2 = BadList()
print(lst2.items) # [1]! Shared default list!

# CORRECT:
def __init__(self, items=None):
    if items is None:
        items = [] # Create new list each time
```

Trap 3: Shallow Copy with Nested Objects

```
class Node {
    Node next;
    int value;
}

Node n1 = new Node();
n1.value = 1;
Node n2 = new Node();
n2.value = 2;
n1.next = n2;

// Shallow copy of n1
Node copy = new Node();
copy.value = n1.value;
copy.next = n1.next; // copy.next points to SAME n2!
```

```
// Now modifying n2 affects both original and copy!
```

Trap 4: Forgetting to Override equals()

```
class Point {
    int x, y;
    // No equals() override!
}

Point p1 = new Point(1, 2);
Point p2 = new Point(1, 2);

// These are both false!
System.out.println(p1 == p2);      // false (different objects)
System.out.println(p1.equals(p2)); // false (inherited == checks references!)
```

[WARNING] **Professor Trap:** If you override equals(), also override hashCode() for Maps/Sets!

Examples

```
class Point {
    private int x, y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true; // Same reference
        if (obj == null || getClass() != obj.getClass()) return false;
        Point other = (Point) obj;
        return x == other.x && y == other.y;
    }

    @Override
    public int hashCode() {
        return 31 * x + y; // Consistent with equals()
    }
}

// Now:
Point p1 = new Point(1, 2);
Point p2 = new Point(1, 2);
System.out.println(p1.equals(p2)); // true! Same values!
System.out.println(p1 == p2);      // false! Different objects!
```

Example 1: Proper equals() Implementation

```
class Person implements Cloneable {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    protected Object clone() {
        return new Person(this.name, this.age); // Deep copy
    }
}

Person original = new Person("Alice", 25);
Person cloned = (Person) original.clone();

System.out.println(original == cloned); // false
System.out.println(original.name.equals(cloned.name)); // true
```

Example 2: Clone Method

1.3 Testing

Testing Concepts

Why test? - Find bugs before users do - Verify functionality works correctly - Enable confident refactoring - Document expected behavior

Unit Testing Basics

```
// Simple unit test example (JUnit)
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class Calculator {
    int add(int a, int b) { return a + b; }
    int divide(int a, int b) {
        if (b == 0) throw new ArithmeticException("Division by zero");
        return a / b;
    }
}
```

```

class CalculatorTest {
    @Test
    void testAdd() {
        Calculator calc = new Calculator();
        assertEquals(5, calc.add(2, 3));
        assertEquals(0, calc.add(-1, 1));
    }

    @Test
    void testDivide() {
        Calculator calc = new Calculator();
        assertEquals(2, calc.divide(10, 5));
    }

    @Test
    void testDivideByZero() {
        Calculator calc = new Calculator();
        assertThrows(ArithmeticException.class, () -> calc.divide(1, 0));
    }
}

```

Testing Terminology

Term	Meaning
Unit test	Tests single method/class
Integration test	Tests multiple components together
Test fixture	Setup code before tests
Assertion	Check that expected == actual
Mock	Fake object for testing dependencies
TDD	Test-Driven Development (write tests first)

White-Board Example: Testing OOP Concepts

Question: What is the output?

```

Dog d1 = new Dog("Buddy", 3);
Dog d2 = d1;           // What is d2?
d2.setAge(4);
System.out.println(d1.getAge());

```

Answer: 4

Explanation:

1. d1 created with age 3
2. d2 = d1 means d2 points to SAME object as d1
3. d2.setAge(4) modifies the shared object

4. `d1.getAge()` returns 4

This tests understanding of REFERENCE semantics!

Practice Questions

MCQ 1: What does `new` do in Java? - A) Declares a new variable - B) Allocates memory and calls constructor - C) Copies an existing object - D) Deletes an object

MCQ 2: Two objects created with `new Dog()` and same values - what is `==`? - A) true - B) false - C) Depends on `equals()` method - D) Compile error

MCQ 3: What is shallow copy? - A) Copies all fields including references - B) Copies only primitive fields - C) Creates new referenced objects - D) Used for value types only

Short Answer: Explain why `String s1 = new String("hi") == String s2 = new String("hi")` returns false.

Board Coding: Write a class `Circle` with radius field, constructor, and method `getArea()`. Then write tests for it.

Oral Explanation: What's the difference between `==` and `.equals()` for objects? When would you use each?

Solutions

MCQ 1: B) Allocates memory and calls constructor

MCQ 2: B) false - different objects in memory

MCQ 3: A) Copies all fields, but reference fields point to same objects

Short Answer: `new` creates a new object in heap memory. Even though both strings have content "hi", they are different objects at different memory locations, so `==` compares references (addresses) and returns false. Use `.equals()` for content comparison.

Cheat-Sheet

Concept	Key Points
Class	Blueprint/template defining structure and behavior
Object	Instance of class, lives in heap memory
Instantiation	<code>new ClassName()</code> allocates and initializes
Constructor	Initializes new object, has same name as class
Reference	Variable that points to object in heap
null	Reference pointing to nothing
==	Compares references (memory addresses)
equals()	Compares object values (override for custom comparison)
Shallow copy	Copies object, shared references inside
Deep copy	Copies object AND all nested objects

Review Checklist

- ☐ **Class vs Object:** Can you explain the difference?
- ☐ **Instantiation:** Do you understand what **new** does?
- ☐ **Reference semantics:** Can you trace reference assignments?
- ☐ **Comparison:** Do you know when to use `==` vs `equals()`?
- ☐ **Copying:** Can you explain shallow vs deep copy?
- ☐ **Testing:** Can you write basic unit tests?